

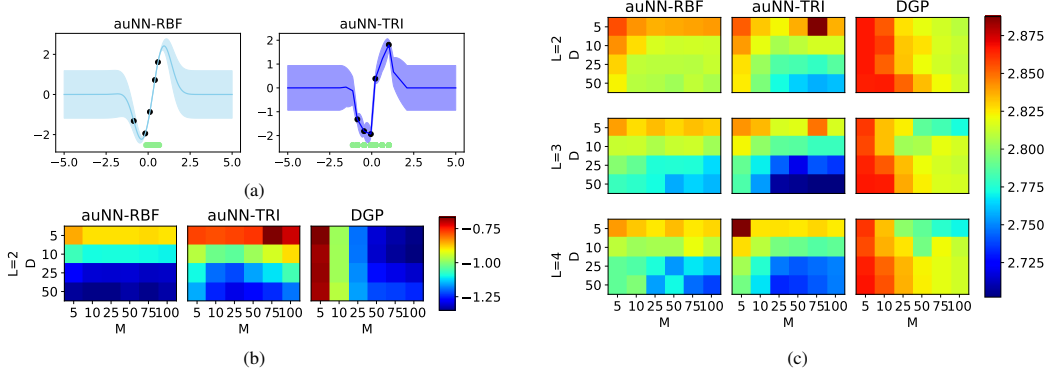


Figure 6: **(a)** One example of activation function (mean and standard deviation) learned by auNN with each kernel. RBF’s one is smoother, whereas TRI’s is piecewise linear, inspired by ReLu. Black dots represent (the mean of) the inducing point values. Green dots are the locations of input data when propagated to the corresponding unit. **(b)** Test NLL of auNN and DGP for different values of M (number of inducing points) and D (number of hidden units). The lower the better. The results are the average over five independent runs with different splits. Whereas DGP improves “by columns” (i.e. with M), auNN does it “by rows” (i.e. with D). This is as hypothesized, and is convenient from a scalability viewpoint. **(c)** Test NLL with increasing depth ($L = 2, 3, 4$). This supports that auNN might benefit more than DGP from deeper networks. Moreover, the aforementioned different influence of M and D on DGP and auNN is confirmed here.

Moreover, the aforementioned different influence of D and M on DGP and on auNN is also confirmed here. The results on RMSE are similar, see Figure 10 and Tables 11-12 in Appendix C.

Finally, it may be argued that auNN closely resembles a DGP with additive kernel (Duvenaud et al., 2011; Durrande et al., 2011) (DGP-add hereafter). Recall that an additive kernel models functions that are decomposed as $f(\mathbf{x}) = f_1(x_1) + \dots + f_D(x_D)$. Therefore, the model for $a^{l+1}|a^l$ in auNN is very similar to that of $f^{l+1}|f^l$ in DGP-add, see Figure 11 in Appendix C. Specifically, in both cases, the input (a^l in auNN, f^l in DGP-add) goes through 1D GPs and then these are aggregated (linear combination through $\mathbf{W}$ in auNN, summation in DGP-add) to yield the output (a^{l+1} in auNN, f^{l+1} in DGP-add). However, there exists a key difference. In auNN, all the nodes in the $(l+1)$ -th layer (i.e. a_i^{l+1}) aggregate a *shared* set of distinct functions (namely, f_i^l), each node using its own weights to aggregate them. While in DGP-add, there is not such shared set of functions, and each node in the $(l+1)$ -th layer (i.e. f_i^{l+1}) aggregates a different set of GP realizations (i.e. the unlabelled blue nodes in Figure 11c). This subtle theoretical difference has empirical implications, since many more functions need to be learned for DGP-add. Indeed, Figures 12 and 13 in Appendix C compare the performance of DGP-add and auNN-RBF (the experimental setting is analogous to that of Figure 6c)⁶. We observe that the results obtained by DGP-add are worse than those by auNN-RBF, probably due to the larger number of functions that need to be learned in DGP-add.

3.4 CLASSIFICATION, SCALABILITY, AND ADDITIONAL METRICS

So far, we have experimented with small to medium regression datasets, and uncertainty estimation has been measured through the (negative) log likelihood and the visual inspection of the predictive distribution (Figures 3 and 5). Here we focus on two large scale classification datasets (up to 10^7 instances), and additional metrics that account for uncertainty calibration are reported. We use the well-known particle physics binary classification sets HIGGS ($N = 11\text{M}$, $D = 28$) and SUSY ($N = 5\text{M}$, $D = 18$) (Baldi et al., 2014). We consider DGP as a baseline, as it obtained state-of-the-art results for these datasets (Salimbeni & Deisenroth, 2017). For all the methods, we consider a Robust-Max classification likelihood (Hernández-Lobato et al., 2011).

The metrics to be used are the Brier score (Gneiting & Raftery, 2007) and the Expected Calibration Error (ECE) (Guo et al., 2017). The former is a proper score function that measures the accuracy of probabilistic predictions for categorical variables. In practice, it is computed as the mean squared

⁶For a fair comparison, here we use auNN-RBF (and not TRI), because DGP-add leverages a RBF kernel.

Table 2: Brier score and expected calibration error (ECE) for auNN and DGP in the large scale classification datasets HIGGS and SUSY (the lower the better in both metrics). The standard error (on three splits) is close to zero in all cases, see Table 13 in Appendix C.

				auNN						DGP		
		N	D	RBF-2	RBF-3	RBF-4	TRI-2	TRI-3	TRI-4	DGP-2	DGP-3	DGP-4
Brier	HIGGS	11M	28	0.3363	0.3159	0.3098	0.3369	0.3172	0.3118	0.4527	0.4399	0.4378
	SUSY	5M	18	0.2746	0.2739	0.2737	0.2749	0.2742	0.2738	0.3815	0.3816	0.3804
ECE	HIGGS	11M	28	0.2196	0.2383	0.2427	0.2198	0.2390	0.2397	0.4352	0.4303	0.4251
	SUSY	5M	18	0.3453	0.3496	0.3504	0.3462	0.3485	0.3465	0.5304	0.5291	0.5273

Table 3: Average training time per batch over 50 independent runs (in seconds). The standard error is low in all cases, see Table 14 in Appendix C.

		auNN						DGP		
		RBF-2	RBF-3	RBF-4	TRI-2	TRI-3	TRI-4	DGP-2	DGP-3	DGP-4
HIGGS		0.0962	0.1607	0.2259	0.0922	0.1647	0.2308	0.1918	0.3102	0.3930
SUSY		0.0926	0.1564	0.2245	0.0923	0.1563	0.2265	0.1430	0.2129	0.2771

difference between a one dimensional vector with the probability for each class label and the one-hot encoding of the actual class. The latter measures miscalibration as the difference in expectation between confidence and accuracy. This is done by partitioning the predictions in M equally spaced bins and taking a weighted average of the bins’ accuracy/confidence difference, see (Guo et al., 2017, Eq.(3)) for details.

Table 2 shows the Brier score and ECE for auNN and DGP for different values of L (depth). We observe that auNN outperforms DGP in both metrics, achieving superior uncertainty estimation. Both TRI and RBF kernels obtain similar results for auNN. Notice that the Brier score generally improves with the network depth, whereas the performance in ECE decreases with depth. Interestingly, this behavior was also observed for standard NNs (Guo et al., 2017, Figure 2a).

Finally, as was theoretically justified in Section 2, auNN can scale up to very large datasets (HIGGS has more than 10^7 training instances). Regarding the practical computational cost, Table 3 shows the average training time per batch for both auNN and DGP in the previous datasets. Although the theoretical complexity is analogous for both methods (recall Section 2), the experiments in Figures 6b-c showed that DGP requires larger values of M , whereas auNN needs larger D ⁷. Since the computational cost is not symmetric on M and D , but significantly cheaper in the latter (recall Section 2), auNN is faster than DGP in practice.

4 RELATED WORK

Activation-level uncertainty is introduced here as an alternative to weight-space stochasticity. The expressiveness of the latter has been recently analyzed in the recent work (Wenzel et al., 2020), where the authors advocate a modified BNN objective. Alternatively, different prior specifications are studied in (Hafner et al., 2020; Pearce et al., 2019; Flam-Shepherd et al., 2017), in addition to the fBNN discussed here (Sun et al., 2019). However, none of these works consider stochasticity on the activations.

Since we present a straightforward use of VI for auNN, in this work we have compared empirically with the well-known VI-based BBP for BNNs. Yet, we expect auNN to benefit from independent inference refinements like those proposed over the last years for BNNs. For instance, natural-gradient VI allows for leveraging techniques such as BatchNorm or data augmentation (Osawa et al., 2019), and the information contained in the SGD trajectory can be exploited as well (Maddox et al., 2019). Also, getting rid of the gradient variance through deterministic approximate moments has provided enhanced results in BNNs (Wu et al., 2019).

⁷In this section, both DGP and auNN are trained with one hidden layer and their optimal configuration according to the previous experiment: large M for DGP ($M = 100$, D is set as recommended by the authors, i.e. $D = \min(30, D^0)$), and large D for auNN ($D = 50$, M is set to the intermediate value of $M = 25$).

A key aspect of auNN is the modelling of the activation function. This element of neural nets has been analyzed before. For instance, self-normalizing neural nets (Klambauer et al., 2017) induce the normalization that is explicitly performed in related approaches such as BatchNorm (Ioffe & Szegedy, 2015) and weight and layer normalization (Salimans & Kingma, 2016; Ba et al., 2016). Learnable deterministic activations have been explored too, e.g. (He et al., 2015; Agostinelli et al., 2014). However, as opposed to auNN, in all these cases the activations are deterministic.

Probabilistic neural networks such as Natural-Parameter Networks (NPN) (Wang et al., 2016) propagate probability distributions through layers of transformations. Therefore, the values of the activations are also described by probability distributions (specifically, the exponential family is used in NPN). Fast dropout training (Wang & Manning, 2013) and certain variants of NPNs can be also viewed in this way (Shekhovtsov & Flach, 2018; Postels et al., 2019). However, in auNN the activations are modelled themselves as stochastic learnable components that follow a GP prior. Along with the deterministic weights, this provides a conceptually different approach to model uncertainty.

A very preliminary study on GP-based activation functions is proposed in (Urban & van der Smagt, 2018). However, the method is not empirically evaluated, no connection with deep GPs is provided, and the inference approach is limited. Namely, the output of each unit is approximated with a Gaussian whose mean and covariance are computed in closed-form, as was done in (Bui et al., 2016) for DGPs. However, this is only tractable for the RBF kernel (in particular, it cannot leverage the more convenient TRI kernel studied here), and the Gaussian approximation typically yields worse results than Monte Carlo approximations to the ELBO as used here (indeed, DSVI (Salimbeni & Deisenroth, 2017) substantially improved the results for DGPs compared to (Bui et al., 2016)).

5 CONCLUSIONS AND FUTURE WORK

We proposed a novel approach for uncertainty estimation in neural network architectures. Whereas previous methods are mostly based on a Bayesian treatment of the weights, here we move the stochasticity to the activation functions, which are modelled with a simple 1D GP and a triangular kernel inspired by the ReLu. Our experiments show that the proposed method obtains better calibrated uncertainty estimates and is competitive in standard prediction tasks. Moreover, the connection with deep GPs is analyzed. Namely, our approach requires fewer inducing points and is better suited for deep architectures, achieving superior performance.

We hope this work raises interest on alternative approaches to model uncertainty in neural networks. One of the main directions of future research is to deeply understand the properties induced by each one of the kernels considered here (i.e. the triangular one and RBF). In particular, it would be interesting to automatically learn the optimal kernel for each unit in a probabilistic way. Also, the use of a GP prior for the activation function may hamper the scalability of auNN to wider and/or deeper networks. In these cases, the GP-based activation model could be substituted by a simpler Bayesian parametric one. This would allow for a cheaper modelling of uncertainty within the activations. Finally, since only the activation function is modified, important deep learning elements such as convolutional layers can be still incorporated.

ACKNOWLEDGEMENTS

This work was supported by the “Agencia Estatal de Investigación” of the Spanish “Ministerio de Ciencia e Innovación” under contract PID2019-105142RB-C22/AEI/10.13039/501100011033, and the Spanish “Ministerio de Economía, Industria y Competitividad” under contract DPI2016-77869-C2-2-R. DHL acknowledges support from the Spanish “Ministerio de Ciencia e Innovación” (projects TIN2016-76406-P and PID2019-106827GB-I00/AEI/10.13039/501100011033). PMA was funded by La Caixa Banking Foundation (ID 100010434, Barcelona, Spain) through La Caixa Fellowship for Doctoral Studies LCF/BQ/ES17/11600011, and the University of Granada through the program “Proyectos de Investigación Precompetitivos para Jóvenes Investigadores del Plan Propio 2019” (ref. PPJIB2019-03).

REFERENCES

F. Agostinelli, M. Hoffman, P. Sadowski, and P. Baldi. Learning activation functions to improve deep neural networks. *arXiv preprint arXiv:1412.6830*, 2014.